In this case, WebDriver automatically instantiates the `email` and `password` fields ❶. For the `email` field, for example, this is the equivalent of first trying `@FindBy(id="email")`, and if that fails, `@FindBy(name="email")`.

The `@FindBy` annotation isn't limited to individual fields; you can also use this notation to retrieve collections of web elements. All you need to do is define your field as a list of `WebElements` instead of a simple `WebElement`, as shown here:

```
@FindBy(css = ".featured .destination-title")
private List<WebElement> featuredDestinations;
```

Match a list of destination titles on the web page.

Each WebElement represents an individual title.

WebDriver also provides the `@FindBys` annotation, which can be used to define nested `@FindBy` annotations.

```
@FindBys({@FindBy(id="main-navbar"), @FindBy(linkText = "Book")})
WebElement bookMenu;
```

This is the equivalent of the nested `findElement()` methods we saw in section 8.2.2:

```
WebElement bookMenu = driver.findElement(By.id("main-navbar"))
                  .findElement(By.linkText("Book"))
```

By default, if WebDriver can't find an element on the page to match a WebElement field in your page object, it will fail immediately. This is often a good thing, because if WebDriver can't find an element it was expecting to find, then either your test or the application is likely to be broken, and you should be notified about this as quickly as possible. But if you're working with dynamic pages using AJAX-based web elements, you might need to give the application some time for a dynamic field to appear before failing the test.

The type-ahead feature we looked at earlier is a good example of this. You could use an `@FindBy` annotation like the following to retrieve these values. The corresponding page object might look something like this:

```
public class BookingPage {
    @FindBy(css = ".typeahead li")
    private List<WebElement> typeaheadEntries;

    public BookingPage(WebDriver driver) {
      PageFactory.initElements(driver, this);
    }
    public List<String> getTypeaheadEntries() {
        List<String> entries = new ArrayList<String>();
        for(WebElement typeaheadElement : typeaheadEntries) {
            entries.add(typeaheadElement.getText());
        }
        return entries;
    }
    ...
}
```

Type-ahead entries will be stored here.

Initialize the typeaheadEntries field.

❶ **Extract the text contents of the type-ahead web elements and return them as a list of Strings.**